

Design Procedure

There were three tasks in this lab. We approached all of them straight forward as all three tasks had instructions of how to get started. For task 1 it was simple of going to the lp catalogue and getting the needed ram 32x3 and instantiating it, and then we made the testbench. We followed the same logic of creating the multidimensional array and following logic for task 2. Finally, we outlined what changes and new files we would need before adding and editing our top module in task 3.

Task #1 Memory Using Library Modules

For the implementation of this lab, we imported a RAM 1-Port .v module from the IP catalog in Quartus. We then instantiated it under a module 'task1' shown in in Figure 1 below. This allowed us to read from a selection of 32 addresses and write data of three bits to the address. There is a write enable signal that also allows us to write vs read from the module.

```
//Shomik Sen Kavin Sundar
//4/17/26
//LAB2
//synopsys translate_off
//timescale 1 ps / 1 ps
//synopsys translate_on
//Task 1 module that calls the 32x3 1 port ram
module task1(input logic clock, input logic [4:0] address, input logic [2:0] data, input logic wren, output logic [2:0] q);
    ram32x3 ram(.clock(clock),.address(address),.data(data),.wren(wren),.q(q));
endmodule
```

Figure 1: Initialization of ram32x3 from IP catalog

Task #1&2 Modelsim

In Figure 2 below, we are testing the ram modules from task1 and task2. For the first 100ps, we are writing data to a specific address then reading from each address to see if the data is properly stored. It accurately shows that the output q is the data that is read from the specific address. After 1000ps, we iterate through each address and write 1 to each pf them. Then we read through each address and see that the output q is 1.

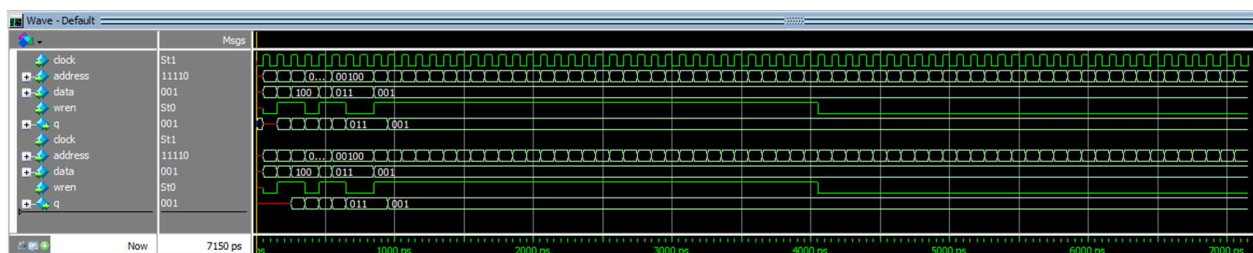


Figure 2: Modelsim for task1 and task2

Task #2 Memory Using SystemVerilog

Figure 3 below shows the module for our hand built memory. This memory has a width of 3 bits (data) a depth of 32 bits (address). If the wren is enabled the memory will update at the desired address and read the data as well. Otherwise, it will just read the data.

```
module task2 (input logic clock, input logic [4:0] address, input logic [2:0] data, input logic wren, output logic [2:0] q);  
    logic [2:0] memory_array[31:0];  
    always_ff @(posedge clock) begin  
        if(wren) begin  
            memory_array[address] <= data;  
            q <= data;  
        end  
        else  
            q <= memory_array[address];  
        end  
    end  
endmodule
```

Figure 3: task2 memory module

Task #3 Library Memory with Independent Read and Write

For task3 we had independent read and write addresses. When SW[9] is enabled, the address that the data is reading from is the addresses associated with the MIF file. The write address is the address to the ram32x3port2 file that the data is being written to. The model Sim for this is combined with the top level modalism in Figure 8 below.

Task #3 Helper Method (One_sec_counter) & Counter

This module creates an enable signal that pulses every one second related to the CLOCK_50 signal. This allows us to create a counter that increments roughly around 1 second. When the enable signal is high it will count upwards to 31. This is wired into our task3 module's read address to increment which address the data is coming from. In the bottom of Figure 8 below, the waveforms for this module can be found as it is wider than the CLOCK_50 pulses.

```
module one_sec_timer #(parameter COUNT_MAX = 550000)(  
    input logic clk,  
    input logic reset,  
    output logic oneSec);  
    logic [$clog2(COUNT_MAX):0] count; //Makes count t  
    always_ff @(posedge clk or posedge reset)begin  
        if(reset)begin  
            count<=0; //makes count and  
            oneSec<=0;  
        end  
        else begin  
            if(count >= COUNT_MAX - 1)begin //checks  
                count<=0;  
                oneSec<=1;  
            end  
            else begin  
                count <= count+1; //increm  
                oneSec<=0;  
            end  
        end  
    end  
endmodule
```

Figure 4: one_sec_timer module

This is the counter module that increments the read address by one second. It will only start the increment when the onesec pulse is triggered.

```
always_ff @(posedge CLOCK_50) begin
    if(reset) begin
        count <= 6'd1;
    end
    else if (onesec) begin
        count <= count + 1;
        if (count >= 6'd33) begin
            count <= 6'd1;
        end
    end
end
```

Figure 5: counter code

MIF File

Here is the MIF file that has the data and addresses store in our ram32x3port2 ram.

Addr	+0	+1	+2	+3	+4	+5	+6	+7	ASCII
0	0	1	2	3	4	5	6	7	
8	7	6	5	4	3	2	1	0	
16	0	2	4	6	2	4	6	0	
24	1	3	5	7	1	3	5	7	

Figure 6: MIF file

Top Level Waveforms

For the first half of the waveforms with the SW9 is low, the data is being read and written to the created ram from task2. We initialized different data numbers and read them from the addresses. This is shown in HEX0 in the beginning where it displays the different data that we are reading from the ram. Next, we enabled SW9 and read the data from the MIF file as the counter increased. As shown in the q output in the middle of our modelsim, in the second half it goes from 2,4,5,0,1,3,5,7.... onwards which are the same as the values from our MIF file from Figure 6.

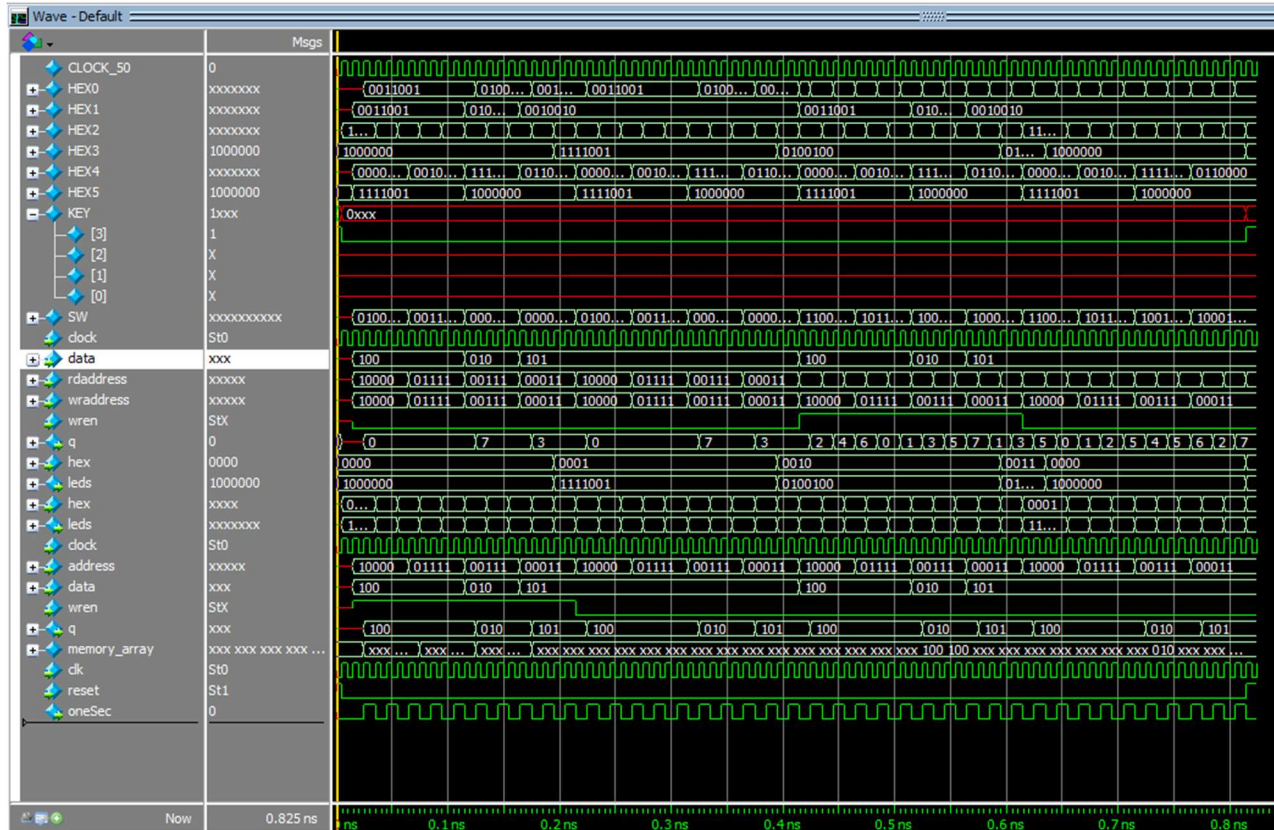


Figure 7: Top Level ModelSim

Overall System: Top Level Block Diagram

The final task was combining all of this into one module. The top level block diagram for our Memory Blocks task2 and task3 memory functions are put into the top-level module. It allows us to drive data to memory created by us from Figure 3 and to memory created using the IP catalog from task3. In the DE1_SoC, SW[9] toggles between task1 and task2 while SW[8:1] determines the address and data to read/write. SW[0] is the write enable and KEY[3] is the reset. The address inputs are wired to ram32x3port2's write address and task2 regular address. There is also a counter module that increments from 0-32 addresses and is used to be the addresses for the ram32x3port2. When SW[9] is enabled, the data will be written to ram32x3port2 and displayed on HEX0. When SW[9] is off, the data will be written to the ram on task2s module. The address from SW[8:4] is displayed on HEX5 and HEX4. The counter which increments each address is displayed on HEX3 and HEX2. HEX1 displays the data that we want to write and HEX0 displays the data that is in the address already.

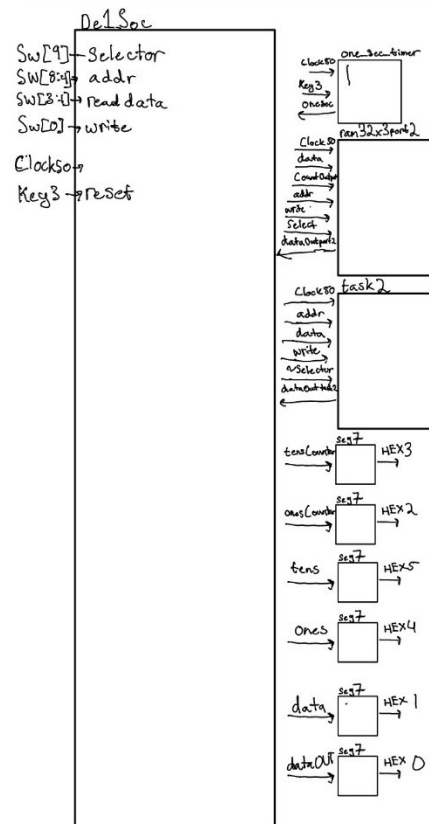


Figure 8: Top Level Block Diagram

Results

Our system ended turning into a system that either uses a multidimensional array (task2) or a 32x3 2 port ram (task3), depending on the user. It can read and write data, with influence from the mif file attached. This system uses a counter and other helper modules like a one second timer, 7 segment display. Finally it outputs the write address, write data, counter, Data In, and Data Out to appropriate HEXs. We decided that reset (key3) only freezes the counter and keeps all the other data paused as to not remove anything that may be useful to user.

Flow Summary

Here is the Flow summary of our top-level design.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Fri Apr 17 15:06:19 202
Quartus Prime Version	17.0.0 Build 595 04/25/2017 SJ Lite
Revision Name	DE1_SoC
Top-level Entity Name	DE1_SoC
Family	Cyclone U
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	N/A
Total registers	28
Total pins	67
Total virtual pins	0
Total block memory bits	192
Total DSP Blocks	0
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0
Total DLLs	0

Figure 11: The ModelSim Flow Summary of compilation of the DE1_SoC module.

Experience Report

Experience ReportAt first, we thought that this lab was going to be straightforward. We completed task 1 and task2 very quickly. However, as we started task3 and began wiring everything and testing on modelsim, we found errors that were not too straightforward. Debugging and finding out what was going wrong with our code using modelsim signals took much longer than anticipated. We ran into so many issues that we did not know how to fix but slowly began pulling signals and analyzing. We learned a great deal about how to debug using modelsim signals. Overall this lab took a long time, however, we overcame much adversity, learned so much, and became better designers in the end.

This lab took us approximately 8 hours, broken down as follows:

- Reading – 1 Hour
- Planning – 30 minutes
 - Design – 1 hour
 - Coding – 4 hours
 - Testing – 3 hours
- Debugging – 7 hours